Design — agent-hands

Record-once / replay-many computer-use automation for legacy banking UIs

Ashish Kosana

19 August 2026

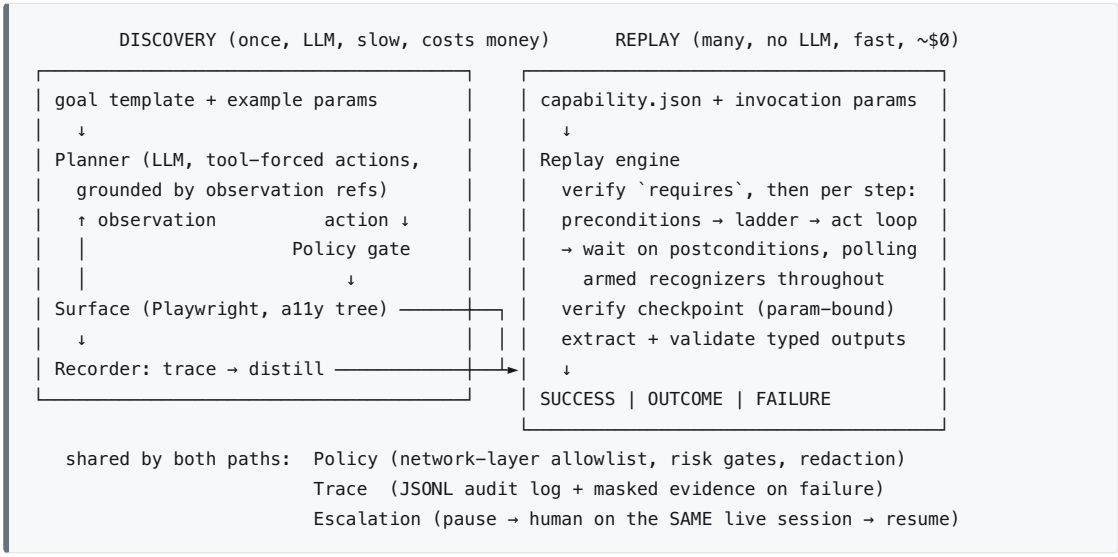
Contents

One line: an LLM figures out a UI flow once (*discovery*); the successful run is distilled into a typed, versioned **capability artifact**; production invocations *replay* that artifact deterministically, with no model in the decision loop.

Five invariants the whole design serves:

1. **Never guess.** Ambiguity at replay is a failure, not a heuristic.
2. **Wrong-but-confident is the worst outcome.** Uniqueness is not correctness; every match and every extracted value is verified against recorded expectations.
3. **A business outcome is an answer, not an error.** The result contract is three-way and the caller can enumerate every code it may receive.
4. **Irreversible actions are never auto-retried and never replayed unreviewed.**
5. **Replay sends nothing to any model, ever** — proven hermetically, not asserted.

Architecture



Components, each independently testable: **Surface** (perceive/act seam), **Planner** (LLM tool-use loop over an OpenAI-compatible client, Groq-hosted by default), **Recorder** (trace → artifact), **Replay engine** (zero model dependency), **Policy**, **Escalation**, **Trace**.

The capability artifact

JSON on disk, Pydantic-modeled, diffable, reviewable. The caller-facing contract is fully declared at the top level — parameters, outputs, **outcomes** — so an agent can invoke it blind; steps are implementation detail below that line.

```
{
  "schema_version": 1,
  "name": "lookup_member_balance",
  "version": 1,
  "description": "Look up a member by ID and read their current savings balance.",
  "target": { "app": "fairview-teller", "entry": { "web": { "url": "http://127.0.0.1:8000/" } } },
  "requires": [
    { "kind": "role_name_visible", "role": "heading", "name": "Fairview Teller Console",
      "context": [ { "name": "main" } ] } ],
  "parameters": { "member_id": {
      "type": "string", "description": "Member number as printed on statements",
      "sensitive": false, "example": "12345", "pattern": "[0-9]{1,10}" } },
  "outcomes": { "MEMBER_NOT_FOUND": { "description": "No member exists with this ID." } },
  "outputs": { "savings_balance": {
      "type": "decimal", "sensitive": true, "region": "accounts",
      "from": { "ladder": [ { "strategy": "relative", "relation": "cell_right",
                              "anchor": { "strategy": "text", "text": "Savings" } } ],
                "context": [ { "name": "main" } ] },
      "parse": { "kind": "money", "locale": "en_US" } } },
  "regions": {
    "results": { "ladder": [ { "strategy": "relative", "relation": "container_of",
        "container": "table", "anchor": { "strategy": "text", "text": "Search Results" } } ],
      "context": [ { "name": "main" } ] },
    "member_header": { "ladder": [ { "strategy": "relative", "relation": "container_of",
        "container": "table", "anchor": { "strategy": "text", "text": "Member #" } } ],
      "context": [ { "name": "main" } ] },
    "accounts": { "ladder": [ { "strategy": "relative", "relation": "container_of",
        "container": "table", "anchor": { "strategy": "text", "text": "Account" } } ],
      "context": [ { "name": "main" } ] } },
  "steps": [
    { "id": "s1", "intent": "Enter the member ID in the search field",
      "action": { "kind": "type", "text": "{param:member_id}" },
      "target": {
        "ladder": [
          { "strategy": "label", "text": "Member ID" },
          { "strategy": "relative", "relation": "nearest_input_right",
            "anchor": { "strategy": "text", "text": "Member ID" } } ],
        "context": [ { "name": "main" } ],
        "fingerprint": { "role": "textbox", "editable": true } },
      "pre": [ { "kind": "editable" } ],
      "post": [ { "kind": "value_matches_param", "param": "member_id", "normalize": "digits" } ],
      "risk": "safe" },
    { "id": "s2", "intent": "Submit the member search",
      "action": { "kind": "click" },
      "target": { "ladder": [ { "strategy": "role", "role": "button", "name": "Search" } ],
                  "context": [ { "name": "main" } ], "fingerprint": { "role": "button" } },
      "pre": [ { "kind": "visible" } ],
      "post": [ { "kind": "role_name_visible", "role": "heading", "name": "Member Details",
                  "context": [ { "name": "main" } ] } ],
      "risk": "safe" } ],
  "conditions": [
    { "id": "not_found", "armed_after": "s2",
      "match": { "kind": "region_text", "region": "results", "patterns": [ "No member
        matches" ] },
      "classify": "business_outcome", "outcome_code": "MEMBER_NOT_FOUND",
      "provenance": "authored", "verified_by_eval": false } ],
  "checkpoint": { "all": [
      { "kind": "role_name_visible", "role": "heading", "name": "Member Details",
        "context": [ { "name": "main" } ] },
      { "kind": "region_text_matches_param", "region": "member_header", "param":
        "member_id" } ] },
  "allow_unbound_checkpoint": false,
  "risk_review": { "reviewed_by": null, "artifact_hash": null }
}
```

Schema decisions an interviewer will probe, answered:

- **Outcome codes are top-level and closed.** Conditions reference them; the loader validates closure (every `outcome_code` declared, every declared code reachable). Return envelope: `SUCCESS{outputs} | OUTCOME{code, evidence} | FAILURE{report}` — outputs present iff SUCCESS.
- **The checkpoint must bind identity.** Checkpoint conditions accept `{param}` references; load-time validation refuses a capability whose checkpoint references no parameter (an explicit waiver field exists for genuinely parameter-free capabilities). Identity matching is boundary-anchored, never bare substring containment — member "123" must not pass against a page showing member "12345". "Member Details is visible" proves you reached *a* details page; only "member-header matches `{member_id}`" proves it's the right one. Output extraction anchors inside the identity-verified region.
- **Conditions are scoped, structural, and provenance-tracked.** Each is armed for a step range (`armed_after`), matches against a named region or role — never bare text anywhere in the page — and records where it came from (`discovered` | `authored`) plus whether an eval has exercised it. A stale or incidental text match returning a confident wrong answer is the failure class this schema exists to prevent.
- **Typed outputs carry parse specs.** `"decimal"` alone can't turn "$1,234.50" (or "(1,234.50)", or a European tenant's "1.234,50") into a number deterministically. A named parser + locale does; the tenant overlay can override locale. The parser is strict for its declared locale: a comma-decimal rendering *fails* under en_US rather than silently parsing to a 100×-wrong number, and `NaN` / `Infinity` (which a naive Decimal() accepts) are rejected. A parse failure is a FAILURE, never a value.
- **Named regions are declared in the artifact.** Recognizers, identity checkpoints, and output extraction all reference regions by name; the `regions` registry maps each name to its own locator ladder, so "structural scoping" is itself reviewable rather than hard-coded.
- **Version semantics:** a change to parameters, outputs, or outcome codes is a contract change and bumps `version` (callers pin `name@version`); step/ladder edits are revisions within it. Every trace logs the hash of the effective artifact (base + overlay) that actually ran.
- **`requires`** declares session/state preconditions (auth, entry state) in the same condition language, checked before step 1. Unmet →

`PRECONDITION_FAILED`, a distinct result — not a misleading locator failure on a login page.

Locator ladder

Ordered strategies per target; semantics pinned in `schema_version`: matching is exact (whitespace-normalized), uniqueness is counted over **visible** elements only, and **0 matches → next rung; >1 matches → hard failure** (lower rungs are less specific — they are fallbacks, not disambiguators).

1. **role + accessible name**, scoped by context path — reliable for buttons, links, and text-bearing cells. *Honest limit:* a bare `<input>` in a table cell has an **empty** accessible name; roles are always computed, names are not. On legacy form controls the ladder lives on rungs 2–4.
2. **label** — real label association only (`label[for]`, aria-label). Proximity-based labeling is deliberately a *separate* rung (relative, below) so telemetry shows which mechanism actually carried each step on legacy markup, where `<label for>` rarely exists.
3. exact visible **text** (links/buttons); when exact text nests (`<td><b>`), the innermost match wins — nesting is one rendered string, not ambiguity.
4. **relative** — geometric relation to an anchor found by 1–3 ("nearest input right of 'Member ID'", "cell right of 'Savings'"). Defined geometrically (vertical overlap for "same row", not center distance — a header cell one row up must not qualify), not by DOM traversal — that is what makes it both table-soup-proof and desktop-portable. A geometric tie is ambiguity and fails; DOM order never breaks a tie.
5. recorded **CSS** path, web-only, last resort: use emits a fragility warning into drift telemetry, and the rung is **disabled entirely when the replay tenant differs from the recording tenant** (a positional path on a different DOM resolves *uniquely to the wrong element* — uniqueness is not correctness).

Every target also records a **fingerprint** (expected role, editability) captured at distillation; the resolved element is verified against it on every replay — any mismatch fails the resolution outright. (Stricter than an earlier draft's warning-on-safe-steps: acting on a wrong element is never acceptable, whatever the step's risk class.) At distillation, every rung is round-tripped through the same locator engine replay uses, against the record-time page; a rung that can't round-trip is a distillation error, not a latent replay bug. Observation and resolution share one role-computation engine (Chromium's native accessibility tree demotes layout tables; Playwright's role engine follows HTML-AAM — mixing them would make discovery-grounded targets unresolvable at replay on

exactly the markup we care about). Context paths are recorded as name + URL pattern + ordinal per segment and re-resolved from the page root on every poll — handles are never cached across polls, and a detached-frame error during a poll means "state changing, re-observe", not failure.

Record path (discovery)

Input: goal *template* + concrete example params. The observation presented to the planner lists elements with stable **refs**; the action tool schema forces every action to reference a ref plus intent — the model cannot act on anything it can't point to. Loop: observe → propose → policy gate → execute → repeat, until `done(checkpoint_evidence)`, **`outcome(code, evidence)`** (a legitimate non-success ending — "no such member" is an answer at discovery time too), `stuck(reason)`, or limits. Ambiguity is negotiable at discovery (the planner disambiguates; the recorder captures the disambiguation as a relative rung) and fatal at replay.

Distillation (every replacement is listed for human confirmation):

- **Parameter substitution over a closed set of slots** — typed text, select values, target names/anchor text on every rung, condition patterns, checkpoint and postcondition values. Whole-slot or token-boundary matches only, longest value first. A target whose accessible name derives from a param ("12345 — Pat Ruiz") becomes a parameterized name or a structural target; a param occurrence anywhere else is a distillation error.
- **Data-independence check**: distilled conditions may only use stable page chrome (headings, labels, buttons, URL patterns). Observed values that vary with the example data — the member's name, a balance — are rejected from conditions/checkpoints and belong only in declared outputs.
- Pre/postconditions from observed state, with type-aware normalization (UIs reformat: "1000" → "1,000.00"; normalized matches are flagged for confirmation, not silently accepted).
- Checkpoint from `done` evidence, subject to the identity-binding rule.
- Risk marks (see Safety — the planner can only *raise* risk, never lower it).

Recognizer provenance: a happy-path run never sees a not-found page, so recognizers can't come from it. They come from (a) additional short discovery runs per declared outcome — run discovery once with a member ID known not to exist; the terminal observation distills into the recognizer, param-substituted; (b) hand-authoring, in which case the condition carries `provenance: "authored"` and `verified_by_eval: false` until an eval scenario exercises it

— the eval table reports verified vs unverified counts. The growth loop in production: every FAILURE on an unrecognized blocking state carries exactly the evidence a human needs to promote it into a recognizer or overlay entry.

The full (redacted) transcript is evidence in `runs/`; the artifact is a contract. They never mix.

Replay path (production)

Input: artifact + params, validated against the parameter schema; `requires` verified; then per step:

- **Two-level timeout model.** Playwright's own auto-waiting actions get short per-attempt timeouts (1–2 s); the engine owns a per-step budget and runs the act as a bounded retry loop, evaluating armed recognizers *before the first attempt and between attempts* — a session-expiry dialog that appears between steps is recognized and recovered, not smashed into a 30 s TimeoutError.
- **Freshness and precedence.** Recognizers already matching *before* a step's action are suppressed for that step — stale state may not classify a new action — and the suppression lifts the moment the stale state is observed gone, so a genuine new match of the same recognizer still counts. If a postcondition and a recognizer are simultaneously true, the postcondition wins. Every OUTCOME carries the matched node's role/region context as auditable evidence.
- **Recovery semantics.** Recovery steps use the full step schema (ladders, risk labels) and pass the same policy gate; recognizer matching is disabled during recovery (one level, no nesting); each condition declares its resume point (`retry_current_step` | `restart_from: k`) because "continue" is wrong whenever recovery navigates; caps are per-condition-per-run; a failed or exhausted recovery **promotes to FAILURE** naming the original condition.
- **No double fire.** Irreversible steps are never auto-retried — on timeout they escalate. Before *any* re-act, the engine probes for the action's effect (current postconditions, then a forward scan of later steps' conditions and the checkpoint); it re-acts only if the effect is provably absent. "Can your system ever click Submit Transfer twice?" must be answered by structure, not probability.
- Native JS dialogs (`alert`/`confirm`) are invisible to the DOM and auto-dismissed by Playwright unless handled: the Surface registers a dialog handler and exposes them as a first-class condition kind artifacts can

classify; an unmatched native dialog is captured (message + screenshot), deterministically dismissed, and classified FAILURE — or relayed to the operator when attended.

Checkpoint verified (identity-bound), outputs extracted from the verified region and validated against their parse spec.

Zero-LLM proof, hermetic: the proof test runs a full replay in a fresh subprocess with no API keys in the environment and network egress blocked at the socket level except the fixture host, asserting success and zero model events in the trace. (An in-process “module never imported” check survives only as a labeled smoke test — it’s name-brittle and order-dependent under pytest.) The egress block doubles as a test of the Policy allowlist, which enforces the same boundary in production.

Failure taxonomy

CLASS	MEANING	EXAMPLES
`BUSINESS_OUTCOME`	a legitimate answer, declared in the contract	member not found, permission denied, validation rejected
recoverable (internal)	handled, logged, capped; promotes to FAILURE on exhaustion	known interstitial, transient slow load, session-expiry continue
`FAILURE`	stop loudly with a debuggable report	locator not found/ambiguous, fingerprint mismatch, postcondition timeout, unexpected/native dialog, parse failure, recovery exhaustion
`PRECONDITION_FAILED`	the environment, not the flow	not authenticated, wrong entry state
`POLICY_VIOLATION`	the guardrails, not the flow	off-allowlist navigation, unapproved irreversible step

FAILURE reports carry: step id, intent, expected (rendered human-readably), observed, and masked evidence (screenshot + a11y snapshot).

Escalation & control transfer

Triggers: planner `stuck` (discovery); unrecognized blocking state or irreversible step awaiting approval (replay); operator-initiated takeover.

Token protocol. The token is engine-owned in-memory state. The operator console (an HTTP thread in the same process) never touches Playwright — the sync API is thread-affine — it only posts transition *requests*. The engine polls the request flag on every tick of its existing poll loop and at every step boundary; on a pending pause it parks, discarding any in-flight wait’s result.

The console shows control as granted only after the engine acknowledges parking — acknowledgment latency is bounded by one poll interval. Two drivers on one session is structurally impossible, not discouraged.

State machine: `AUTOMATION → PAUSED → {HUMAN | AUTOMATION(approve) | FAILED(deny/TTL)}`; `HUMAN → {AUTOMATION(hand back) | ABORTED(reason) | RESOLVED(outcome_code, note)}`. The human can approve an irreversible step without taking over, abort a run they judge unsafe, or resolve it as a business outcome they established manually — with attribution. An unanswered intervention has a TTL: on expiry the run fails carrying the intervention context, and the browser session is closed (no keep-alive games with a banking session). Every transition records the operator identity string.

Intervention request carries: capability + version, step id and intent, non-sensitive resolved params, why it stopped, the step's postconditions rendered as "when done, the page should show…", the last N trace events, a masked screenshot, and remaining steps. Sensitive values are never displayed; credential steps are performed by the operator with their own credentials.

Human-action capture is installed once at context creation (`add_init_script` + `expose_binding`, re-injected per document and frame — listeners injected at takeover time die on the first navigation), dormant except in HUMAN state. It records **semantic targets only, never values** — "typed 12 chars (masked) into textbox 'Password' in frame main" — for *every* field, because on legacy surfaces SSNs live in plain-text inputs and no heuristic finds them all. Browser-level events (navigation, popup, download, new tab) are captured via Playwright events; on hand-back, if the active page differs from the engine's handle, the engine adopts it explicitly or fails with a clear report. All screenshots during the HUMAN window are suppressed. JS dialogs during HUMAN state are relayed to the console with Accept/Dismiss buttons (Playwright intercepts them; the human cannot click a native dialog on a driven page — a known constraint, handled, not discovered in the demo). Human data entry is evidence, not a replayable macro: if the workaround should become automation, discovery re-runs through that path. During discovery, the token gates the whole loop iteration (observe + LLM call + act), captured human steps distill into artifact steps flagged `origin: "human"` that force review before the artifact is accepted, and on resume the planner's transcript gets a synthetic message: "a human intervened, performed X, current observation is Y."

Resume is a forward scan, not a single-step check. The common case is a human who finishes the screen they're on. On hand-back: validate the current URL against the allowlist (distinct human-attributed outcome if off-list); check the capability checkpoint first — if it holds, skip to output extraction; otherwise scan forward from the current step for the furthest consistent pre/postcondition frontier and resume there; if no unique resume point exists, re-escalate with an operator affordance ("I completed it" / "retry from here"). Never guess — the same rule the ladder enforces.

Safety

- **Allowlist enforced at the network layer**, not in prompts: `context.route` aborts off-allowlist requests (covers redirects, popups, and iframes, context-wide); a navigation listener converts off-list top-level navigation into `POLICY_VIOLATION`; unexpected popups are closed or escalated; frames with off-allowlist origins are excluded from observations so foreign content never reaches a prompt or trace. The allowlist lives in policy config owned by the Policy component — never in the tenant overlay (a tenancy mechanism must not be able to widen a safety boundary).
- **Risk posture is inverted: mutating-by-default.** Any action observed to provoke a non-GET request during discovery defaults to `risky`; planner classification and keyword rules can only *raise* risk, never lower it. A capability is ineligible for unattended replay until a human has reviewed the diffable step list and signed the risk labels (`risk_review` binds to the artifact hash — re-recording invalidates approval). Approval at invocation is per step (`approve: ["s7"]`), not a blanket flag that switches the safety system off. Recovery steps pass the same gates.
- **Secrets are symbolic end-to-end.** Type-actions carry `{param:name}` or `{secret:ENV_VAR}`; the Surface resolves them at act time from an in-process store; the planner receives parameter *names* (with sensitivity flags and non-sensitive examples), never sensitive values, so a secret cannot transit a prompt. A sensitive parameter forbids `example`; sensitive postconditions are evaluated inside the Surface and recorded boolean-only.
- **Taint-based masking, by element identity, not string match.** Any element a sensitive step acted on (and its frame, until navigation) is tainted; every subsequent observation masks its value before serialization to prompt or trace — echo-through-the-a11y-tree is the leak string matching misses, and masking by identity avoids corrupting a "$12,345.00" that happens to contain a member ID. Failure/escalation evidence captured under taint gets tainted regions masked in the screenshot (Playwright

`mask`) and scrubbed from the snapshot — masked, not suppressed, so debuggability survives.

- Outputs can be `sensitive` (masked in traces, returned only in the result object). Trace entries default to observation digests, with full snapshots only on FAILURE evidence, scrubbed. URLs in traces pass a query-value scrubber (declared params + configurable key patterns: ssn, member, acct). `runs/` is operational data with a stated retention position, not an archive.
- **Canary test:** a scripted "human" types a sentinel string during handoff; the suite asserts the sentinel appears nowhere under `runs/`.

Heterogeneity & tenancy (design-only, per the brief)

What transfers to desktop (UIA/AX) cleanly: semantic role+name targets, the ladder discipline, condition polling, checkpoints, typed I/O, the control-token escalation model. **What needs per-surface work:** a role/rung mapping layer (ARIA `textbox` ? UIA `Edit`; context path = frame chain on web, window/pane chain on desktop; CSS rung is declared web-only and skipped elsewhere), input synthesis, dialog scoping as context roots, and — honestly — name computation: the browser *computes* accessible names from labels and adjacent text; a custom-drawn Win32 teller app exposes an empty UIA tree, and the realistic fallback there is an OCR/vision rung, rejected as a *primary* mechanism for the web path but acceptable as a flagged last rung on desktop. The schema is surface-neutral now (`context` paths, surface-typed `entry`, geometric `relative`) precisely so this stays a mapping problem, not a rewrite.

Tenant reuse. The overlay is a **typed patch document**, not "synonyms": keyed by artifact addresses (step id + rung, condition id, checkpoint, output id), with explicit merge semantics (override / append / disable) — a renamed label breaks checkpoints and output anchors too, and the overlay must reach them. An overlay pins its base `{capability, version}`; the loader hard-fails on mismatch (silent best-effort merging across a rev is how you break three hundred tenants at once with no signal). Overlays may only map recognizers to already-declared outcome codes — a genuinely new business outcome is a contract change and bumps the base version for every caller. Structural variance (an extra verification step, different page order) is not an overlay's job: that's a capability **variant** sharing the same parameter/output contract, selected by a tenant → (capability, version/variant) resolution table — callers see one stable tool. Overlay entries are born from the escalation loop (a human resolves an unknown state once; the resolution is captured as a proposed entry) and reviewed like artifacts, because overlays alter behavior.

Drift detection, concretely: every replay appends `{capability, version, tenant, per-step rung index, name source: base|overlay, conditions fired}` to a per-capability health file; a threshold rule (rung > 1 carrying > 20% of the last N runs for a tenant) flips a `needs_rediscovery` status that future replays surface and promotion gates read. Re-discovery re-runs the stored goal template and diffs the parameter/output contract to confirm callers are unaffected.

Eval plan

Pytest scenario suite against the local fixture (offline, CI-able):

- happy-path replay with fresh params;
- six injected runtime states — not-found, permission-denied, validation error, session expiry, surprise interstitial, slow load — each asserting the *correct classification*, with recognizer verified/unverified counts reported;
- interstitial appearing **between** steps (the act-phase recognition path);
- element-removed (quality of the FAILURE report);
- double-submit probe: slow confirmation page on an irreversible step — assert escalation, not re-click;
- one drift case (renamed label; rung 2 catches it) — drift is secondary here;
- N=10 replay stability;
- negative policy tests: off-allowlist action blocked; unapproved irreversible step blocked; recovery step marked irreversible blocked unattended;
- redaction canary (sentinel never persists) and the hermetic zero-LLM proof;
- escalation TTL expiry path.

Reported as a table: scenario × expected vs actual classification; discovery vs replay latency; tokens + cost (replay = 0); stability; per-step rung-hit distribution (CSS rung expected at zero). Evidence from the real LLM discovery run committed under `/evidence/`.

Targets & scope honesty

- **Fixture:** “Fairview Teller” — small Flask app, deliberately legacy: table layout, no test IDs, nav iframe, server-rendered, terse non-semantic markup; fault-injection endpoint; fake seeded data.
- **Second tenant:** “Lakeside” — structurally divergent templates (different frame layout, renamed labels, extra interstitial, reordered columns), not a

string-swap skin, so the two-tenant demo exercises rungs 2–4 and the overlay, not a synonym map.

- **Portability leg:** one simple capability against saucedemo.com (built for automation practice) — proves nothing is welded to self-authored markup.

Cut order if time runs short (never both #1 and #2 — that would leave zero generalization evidence): 1) saucedemo leg; 2) Lakeside skin (tenancy falls back to overlay-merge unit tests + this design); 3) operator console degrades from web page to CLI on the same state machine; 4) the drift eval case. **Never cut:** slices 1–3, the escalation state machine, the zero-LLM proof.

Slices (each ends runnable + tested)

1. Fixture app + artifact schema + replay engine driven by a **hand-written** artifact — the production path first, fully testable with no LLM anywhere.
2. Planner + recorder: discovery produces an artifact that slice 1 replays. The real evidence run happens here.
3. Full failure taxonomy: fault injection, recognizers, three-way contract.
4. Escalation & handoff: token protocol, operator console, human-action capture, resume scan.
5. Policy hardening: network-layer allowlist, risk gates, taint redaction.
6. Evals + evidence + README/REPORT + tenant-variant and portability demos.

Considered and rejected

- **Screenshot + coordinate replay** — coordinates are the brittleness the brief warns about, and re-finding targets each run puts a model back into the model-free path. Kept for evidence/escalation context on web; explicitly *not* foreclosed as a last-rung fallback for custom-drawn desktop controls.
- **CSS/XPath as primary locators** — assumes a clean DOM the brief says doesn't exist. Kept as the flagged, same-tenant-only last rung.
- **Transcript-as-artifact** — not typed, not reviewable, not parameterizable.
- **Agent frameworks** — the loop is a couple hundred lines; a framework adds a dependency to defend without adding capability.
- **A database** — JSON files are diffable and reviewable, which is the point.
- **Services/queues** — single process; the brief explicitly does not reward scaling infrastructure.